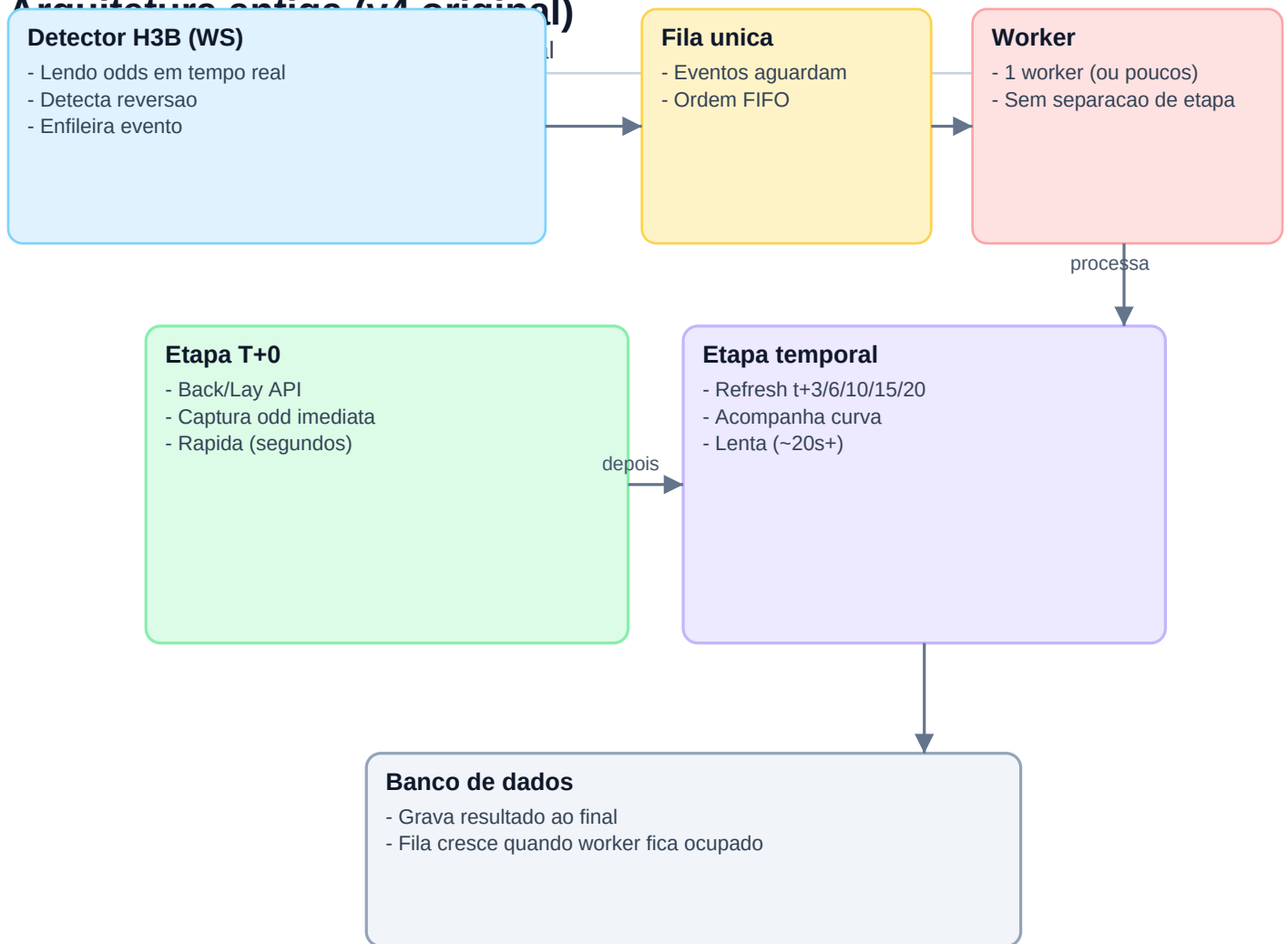


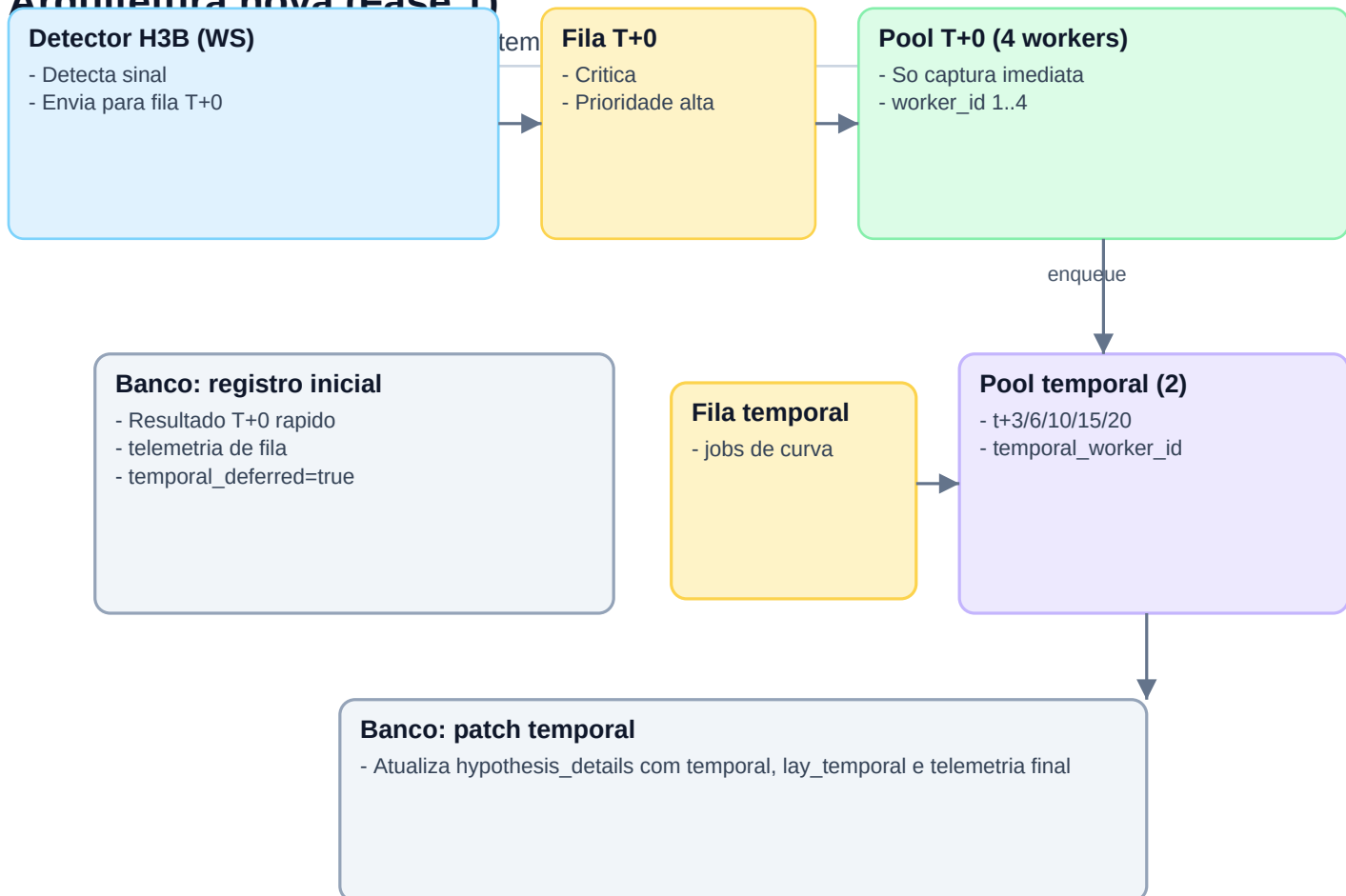
Arquitetura antiga (v1 original)



Problema principal: o mesmo worker fazia a parte rapida (T+0) e a parte longa (temporal). Com rajadas de sinais, o worker ficava ocupado por muito tempo e a fila aumentava.

Consequencia observada antes da mudanca: fila media relevante e espera alta para novos eventos.

Arquitetura nova (Fase 1)



Resultado pratico: T+0 deixa de esperar o temporal. A fila critica cai, e o temporal segue em paralelo. Logs confirmam workers separados e banco mostra worker_id/temporal_worker_id.

Impactos observados e implicacoes

Comparativo antes vs depois da Fase 1 (dados reais da operacao)

Indicador	Antes	Depois (Fase 1)
Queue depth enq (media)	6.93	0.04
Queue depth enq (p90)	14	proximo de 0-1
Queue wait (media)	42.956 ms	0.3 ms
Queue wait (p90)	alto	1.0 ms
Queue wait (p95)	alto	2.3 ms
Cobertura telemetria	parcial	100% na janela
Back/Lay temporal	assimetrico	quase paridade

Recomendacao atual

- Manter 4 workers T+0
- Manter 2 workers temporal
- Reavaliar com janela 12-24h

Gatilhos para proxima mudanca

- Se p95 fila T+0 > 1s: subir T+0
- Se temporal acumular: subir temporal
- Sempre monitorar CPU/RAM/proxy

Proposta de evolucao da arquitetura (proxima versao)

Roadmap simples para estabilidade + latencia baixa

1. Fase 1 (ja ativa)

- Separar T+0 e temporal; pools 4/2; telemetria de fila

2. Fase 2

- Autoajuste de workers por meta (p95 fila T+0 < 1s)

3. Fase 3

- Descartar evento stale no T+0 (evita processar fila velha)

4. Fase 4

- Dashboard com p50/p90/p95/p99 e custo de proxy por worker

Mensagem final: seu objetivo de fila < 1s no caminho critico e viavel. A separacao de esteiras foi o passo estrutural mais importante e ja mostrou ganho real.